

Contents

Coding Agent — a minimal closed-loop control system (ch15 / §17)	1
Two policies, one seam	1
The §17 experiment	1
Layout (derived from SPEC.md §3–§11)	1
CLI quickstart	2
Development setup	2

Coding Agent — a minimal closed-loop control system (ch15 / §17)

A **minimal coding agent**: an LLM embedded inside a deterministic runtime that provides context, tools, state, permissions, execution, and verification. Its cycle is **observe** → **reason** → **act** → **verify** → **feedback** → **repeat** (curriculum §1/§18.2). It instantiates the chapter’s thesis —

The system successfully navigated a software environment to a verified state. — by turning uncertain model output into measurable feedback via an in-loop **verifier**.

The design is the deterministic-vs-probabilistic split carried forward from week 1: a **deterministic core** (sandbox, control loop, context manager, tool controller, permission gate, verifier, instrumenter, reporter — all LLM- and network-free) and a single **probabilistic** boundary, the model, reached *only* through the Policy interface.

Two policies, one seam

- **MockPolicy** — a deterministic offline double (a scripted or rule-driven action sequence). The *entire* automated suite runs over it with **no Ollama and no sockets**, so artifacts are **byte-identical** on the mock path (R-13 / I-002 / K-06).
- **OllamaPolicy** — opt-in, real qwen3.8 over `http://localhost:11434`. Isolated behind Policy (R-15 / I-009); the implementation ships with the lab, but its optional `httpx` dependency is not installed by the default mock-only environment. A missing daemon is resolved by the model-availability taxonomy **DEGRADED MOCK / PULL_REQUIRED / RUN_REAL** (R-14 / E-11 / E-12) so a degraded run is never ambiguous in the trajectory.

The §17 experiment

The core acceptance activity is a **failure-injection** over a pinned fixture (`fixtures/parse-config/`): inject a *canonical defect* into `parse_config`, run the closed loop, and observe that the agent **detects** the FAILED verdict, **diagnoses** the cause, **repairs** it, and the number of iterations to VERIFIED is a *reproducible constant* (3) — not “whatever the model did today” (R-11 / I-013). Every run is instrumented (§17): `iteration`, `tool_calls`, `tokens`, `files_read`, `files_modified`, `tests_executed`, `test_results`, `errors`, `time_ms`, `final_outcome`.

Layout (derived from SPEC.md §3–§11)

```
src/coding_agent/  
  task.py          # C-01 Task input  
  policy.py        # C-02 Policy / MockPolicy / OllamaPolicy + resolve() taxonomy (R-14)  
  permissions.py   # C-04 authorize() - the gate OUTSIDE the model (R-05 / I-008)  
  tools.py         # C-03 ToolController + TOOL_SET, sandbox-bound (I-003)  
  verifier.py      # C-05 VerifySpec / Verdict - closes the loop (R-06 / I-007)  
  instrument.py    # C-06 trajectory iteration row + K-07 deterministic surrogates  
  context.py       # R-03 context engineering + R-10 compaction (char budget, K-05/K-09)  
  sandbox.py       # C-08-bis isolated, ephemeral sandbox + lifecycle (I-003 / I-011)  
  control_loop.py  # R-01/R-08 the observe→...→feedback FSM + stopping conditions
```

```

experiment.py    # R-11 the §17 detect/diagnose/repair experiment
report.py       # C-06/C-07/R-16/R-17 write + schema-gate trajectory/experiment
cli.py          # R-16 the `agent` CLI: run / experiment / inspect / compare
fixtures/parse-config/ # the C-09 pinned unit under test + its verifier
schemas/        # C-06 trajectory.json + C-07 experiment.json (R-17 gate)
tests/          # SPEC §9 behavioral coverage for T-01..T-13 (fully offline)

```

CLI quickstart

All commands below run from this directory. The default path is deterministic and uses the offline MockPolicy; each run copies the repository into an ephemeral sandbox and removes that sandbox when it finishes.

Verify an already-repaired repository; writes trajectory.json.

```

uv run agent run \
--task "parse the configuration" \
--repo fixtures/parse-config \
--mock \
--out trajectory.json

```

Run the pinned §17 failure-injection experiment; writes both artifacts.

```

uv run agent experiment \
--task parse-config \
--repo fixtures/parse-config \
--mock \
--out experiment.json

```

Inspect a saved trajectory without running inference or touching a repository.

```

uv run agent inspect --in trajectory.json

```

Compare two saved trajectories; writes compare_report.json by default.

```

uv run agent compare \
--baseline trajectory.json \
--current trajectory.json

```

Help is a successful command and exits 0.

```

uv run agent --help

```

Use `--script actions.json` with `run --mock` to provide deterministic action batches. A script is a JSON array of batches; each action is either `{"type": "tool", "name": "read_file", "args": {"path": "repo/config.py"}}`, `{"type": "stop"}`, or `{"type": "noop"}`. Use `--sandbox PATH` when a stable artifact path is useful for reproducibility. See SPEC.md §5.1 and §17 for the complete interface and artifact contracts.

Development setup

```

uv sync          # default install: MOCK-ONLY, no model/network client
uv run pytest    # behavioral coverage for SPEC T-01..T-13; fully offline
uv run agent --help # the CLI (R-16), exits 0

```

opt in to the real Ollama policy (adds httpx; still never required by the suite):

```

uv pip install -e '[agents-real]'

```

The authoritative description of behavior, contracts, invariants, edge cases, and acceptance criteria is SPEC.md. Implementation is derived from it; see SPEC.md §11 Traceability for the id → module mapping.